

Question 1. Short Answer Questions

(60 points)

Fill in the box with the answer.

- *Removed in publishable version* -

Question 2. Coding 1

(20 points)

Complete the overridden method `void addFirst(T item)`.

It is a method of the class `DelimitedARDeque`, a subclass of an iterable `ARDeque`.

It adds an item to the front of the object of `DelimitedARDeque`,

but it will throw an `IllegalArgumentException` if the item already occurs in it more than once.

Test case:

```
DelimitedARDeque<String> dd = new DelimitedARDeque<>();
```

```
dd.addFirst("a");
```

```
dd.addFirst("b");
```

```
try {
```

```
    dd.addFirst("a");
```

```
    dd.printDeque();
```

```
} catch (IllegalArgumentException e) {
```

```
    System.out.println("AddFirst failed");
```

```
}
```

→ AddFirst failed

Question 3. Coding 2

(20 points)

Complete the method `boolean equals(Object that)` of an iterable `ARDeque` that overrides the one in `Object`.

To be the same, they must be of the same size, and must share at most 1 different items at the same position.

Suppose `[]` is an `ARDeque<Integer>`. Then:

`[1, 2, 3].equals([1, 2, 5])` → `true`

`[1, 3, 2].equals([1, 2, 3])` → `false`

----- THIS IS THE END OF THE DOCUMENT -----

CPT204 2022-2023 F S2 - Suggested Answers

Note: Question 1 short-answer section is removed in the publishable paper, so only Q2-Q3 can be answered from the available file.

Question 2 Coding 1 - addFirst(T item)

Idea:

- Before adding, count whether item already occurs more than once in the current deque.
- If it already occurs at least once and adding it would make it occur more than once, throw `IllegalArgumentException`.
- Otherwise add it to the front.

Typical answer pattern:

```
public void addFirst(T item) {
    int count = 0;
    for (T x : this) {
        if (item == null ? x == null : item.equals(x)) {
            count++;
        }
    }
    if (count >= 1) {
        throw new IllegalArgumentException();
    }
    super.addFirst(item);
}
```

If the wording is interpreted as "already occurs more than once before adding", change the condition to `count > 1`. The given test case expects the second "a" to fail, so `count >= 1` is the useful implementation.

Question 3 Coding 2 - equals(Object that)

Requirement:

- same type / compatible `ARDeque`
- same size
- at most 1 different item at the same position

Typical answer pattern:

@Override

```
public boolean equals(Object that) {
    if (this == that) return true;
    if (!(that instanceof ARDeque<?>)) return false;

    ARDeque<?> other = (ARDeque<?>) that;
    if (this.size() != other.size()) return false;

    Iterator<T> it1 = this.iterator();
    Iterator<?> it2 = other.iterator();
    int different = 0;

    while (it1.hasNext() && it2.hasNext()) {
        T a = it1.next();
        Object b = it2.next();
        if (!(a == null ? b == null : a.equals(b))) {
            different++;
            if (different > 1) return false;
        }
    }
}
```

```
}    return true;
```